PLaND - Path to Least Non Determinism

Maddipatla Naga Venkata Sai Krishna, Asit Kumar Sahoo

Abstract

Language-model agents often repeat mechanical work because an entire standard operating procedure (SOP) is expressed in natural language. We present Path to Least Non Determinism (PLaND), an evaluation-driven methodology that retains language-model judgment where needed while moving stable steps into references, Python, or Bash. Before measurement, PLaND freezes the model, prompt, harness, data, scorer, seed, runtime settings, permissions, and acceptance rule; only the workflow skill package may evolve. On a balanced top-10, single-label LEDGAR subset, a hybrid SOP passed validation and was evaluated once on 1,000 untouched cases. Accuracy changed from 93.5% to 92.7%, a paired difference of -0.8 percentage points with a bootstrap 95% confidence interval of [-1.6, 0.0]. Tokens fell 40.02%, from 376,088 to 225,573. The hybrid bypassed 411 model calls, and 97.24% of saved tokens came from those bypasses rather than shorter fallback prompts. An optimized post-change replication produced 93.7% versus 92.9% accuracy and the same 411-call reduction. CFPB and SpamAssassin candidates reduced tokens but failed quality gates; three document-image workflows stopped after nonviable baselines. PLaND therefore supports selective, evidence-bound determinisation rather than unconditional conversion to code.

Keywords: *agent skills, deterministic workflows, language-model agents, evaluation, token efficiency, workflow optimization*

Introduction

Language-model agents are valuable when a task requires interpretation, contextual judgment, or exception handling. They are less valuable when repeatedly asked to parse fixed fields, count items, normalize values, validate a schema, or apply an exact rule. Reusable agent instructions are increasingly packaged as skills: the Agent Skills specification requires `SKILL.md` and permits references, scripts, and assets [1]; DeepAgents supports open-ended skill use [2]; and LangGraph represents stable workflows as explicit graphs [3]. These systems do not determine which SOP steps should retain model judgment.

PLaND asks: what is the least non-deterministic form of a workflow that still satisfies its measured quality contract? Natural language remains appropriate for ambiguous decisions. Ordinary code is usually preferable for bounded, testable computation. The intended endpoint may therefore be an open-ended agent, a hybrid skill, or a mostly deterministic graph.

Related systems optimize language-model programs, prompts, workflows, or skills. DSPy compiles declarative model pipelines against metrics [4]; AutoFlow and AFlow generate and optimize workflows [5, 6]; and Automated Design of Agentic Systems searches agent designs [7]. SkillOpt, SkillRevise, SkillReducer, and ACES optimize or evaluate reusable skills [8-11]. PLaND contributes a narrower representation path - English instruction, reference, or command - plus a frozen mutation boundary, paired evaluation, explicit rejection records, and a held-out release gate.

We ask whether a hybrid SOP can reduce model-mediated work while remaining inside a fixed quality margin, whether validation gates block efficient but unsafe candidates, and whether savings arise from fewer calls rather than prompt shortening alone. We do not claim that the current single-run study measures stochastic output variance.

Material and Methods

Methodology and mutation boundary

PLaND uses two open-ended skills. `generate-initial-version` converts natural-language task requirements, approved data sources, and evaluation examples into an initial agent, system prompt, and natural-language SOP. `pland-evolver` uses development traces to propose bounded changes. Task-local runners and scorers define the executable interface and primary quality measure, so the methodology is not restricted to classification or a particular output type.

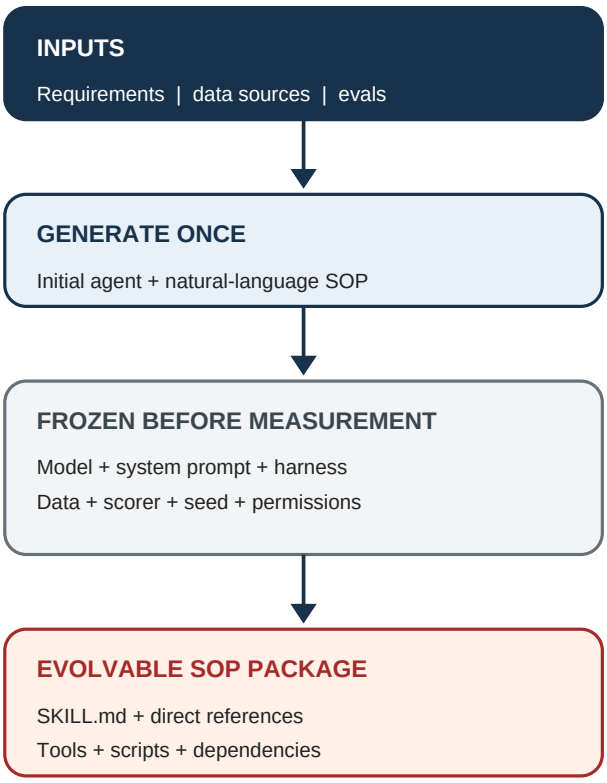

Figure 1. Frozen PLaND boundary. The initial agent and evaluation contract are finalized before baseline measurement; only the workflow skill package may evolve.

Every numbered SOP step is marked as an English instruction, a reference to supporting skill material, or a direct Python/Bash command. A command counts as compiled only when the evaluated runtime invokes it and consumes its output. `SKILL.md` is the SOP source of truth, while the comparison hash also covers directly referenced instructions, invoked scripts, and approved dependencies.

The model and digest, system prompt, normalized harness, evaluation cases and expected outputs, scorer, datasource snapshot, seed, model/runtime settings, permissions, quality floor, non-inferiority margin, and expense objective are frozen. Comparison code rejects mismatched fingerprints, duplicated case identifiers, or missing full hashes. The candidate may change only the skill package. Commands require bounded input/output contracts and an English fallback. Caching and parallelism are allowed only when applied consistently and recorded as runtime invariants.

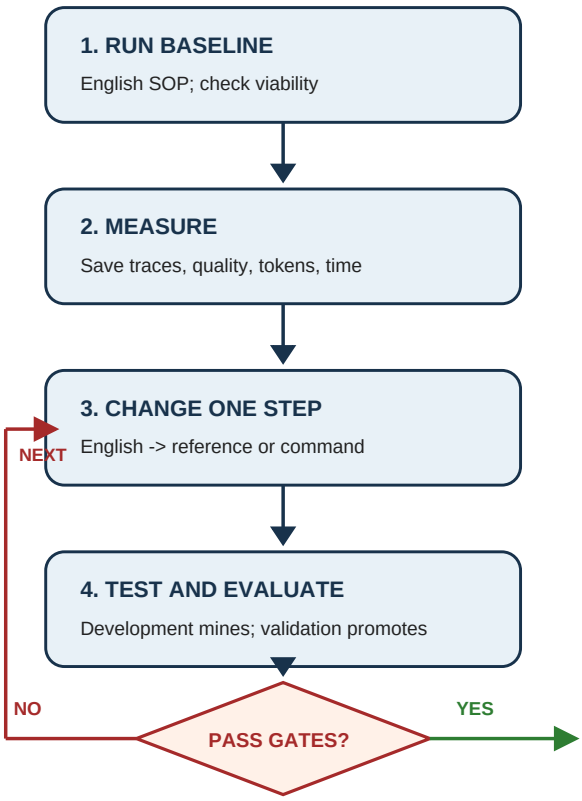

Figure 2. One bounded evolution iteration. Development evidence proposes a change; validation promotes it only when frozen quality and expense gates pass.

Evolution was capped at ten candidates. A baseline first had to meet an absolute viability floor. Candidate selection required task quality within a prespecified margin and strict improvement in one primary expense measure. Development informed changes; validation selected candidates; test cases were released once only after validation passed.

Data and evaluation

We prepared public datasets resembling enterprise work: LEDGAR legal clauses [12], CFPB complaint narratives [13], SpamAssassin email [14], SROIE receipts [15], and RVL-CDIP/Tobacco3482 document images [16, 17]. LiteParse supplied a local OCR condition [18]. Selection manifests record source identifiers and hashes, seed 20260902, exclusions, and split membership. Audits checked unique identifiers, split/content overlap, missing files, label leakage, source boundaries, and pilot overlap.

Table 1. Prepared confirmatory datasets

Workflow	Development / validation / test	Status
LEDGAR clause classification	100 / 100 / 1,000	Prepared and released after validation
CFPB complaint routing	100 / 100 / 1,000	Prepared; test not released
SpamAssassin email classification	100 / 100 / 1,000	Prepared; test not released
QS-OCR/Tobacco3482 classification	100 / 100 / 1,000	Prepared; baseline nonviable
SROIE receipt extraction	100 / 100 / 300	Prepared; baseline nonviable
RVL-CDIP constrained mirror	100 / 100 / 369	Prepared; baseline nonviable

Classification required both variants to reach 0.80 accuracy. Extraction required field F1 of at least 0.50. For baseline b and hybrid h, the lower bound of a paired bootstrap 95% interval for `accuracy_h - accuracy_b` had to be at least -0.02. Token reduction had to be at least 5%, with a positive bootstrap lower bound. We used 5,000 paired resamples with seed 20260902, Wilson accuracy intervals, and an exact McNemar test.

Text experiments used local Ollama 0.33.0 and `qwen3:14b`, digest `bdbd181c33f2ed1b31c972991882db3cf4d192569092138a7d29e973cd9debe8`, on an Apple M5 Pro with 48 GB unified memory. Original confirmatory runs were sequential. The post-change replication disabled thinking and streaming; used temperature 0, seed 20260902, a 4,096-token context, a 128-token output cap, and an exact JSON schema; preloaded and retained the model; and enabled Flash Attention, a q8_0 KV cache, one loaded model, and two parallel requests. Runtime settings, component durations, summed request latency, and elapsed collection time were stored. Parallel elapsed time measures throughput and is not timing-comparable with the sequential study.

Results

Table 2. Primary confirmatory and post-change replication results

Workflow and split	Quality, NL -> hybrid	Tokens, NL -> hybrid	Decision
LEDGAR validation, original	92% -> 93%	37,147 -> 21,104 (-43.19%)	Pass
LEDGAR test, original	93.5% -> 92.7%	376,088 -> 225,573 (-40.02%)	Pass
LEDGAR validation, replication	93% -> 94%	37,148 -> 21,104 (-43.19%)	Pass
LEDGAR test, replication	93.7% -> 92.9%	376,090 -> 225,575 (-40.02%)	Pass
CFPB validation, replication	78% -> 71%	58,372 -> 33,120 (-43.26%)	Reject
SpamAssassin validation, replication	88% -> 84%	188,384 -> 178,881 (-5.04%)	Reject

The original 1,000-case LEDGAR test produced a -0.8-point hybrid-minus-NL accuracy difference (95% interval [-1.6, 0.0]) and a token-reduction interval of [37.00%, 43.08%]. Twelve cases were correct only under NL, four only under hybrid, 923 under both, and 61 under neither; exact McNemar p = 0.0768. The result satisfies the chosen non-inferiority contract, not equivalence.

Model calls fell from 1,000 to 589 because 411 cases used deterministic commands. Those cases accounted for 146,366 of 150,515 saved tokens (97.24%); shorter prompts on the 589 fallback cases saved only 4,149 tokens. Accuracy on the command-routed subset was 97.08% for NL and 96.11% for hybrid. Thus most expense reduction came from avoiding model calls while the overall task remained inside the frozen quality margin. The replication reproduced the -0.8-point accuracy difference, the 40.02% token reduction, and the 411-call bypass. It is supporting evidence, not a new untouched test.

CFPB demonstrates the rejection mechanism. The replication saved 43.26% of tokens, but accuracy fell from 78% to 71%; the paired accuracy-difference interval was [-13, -2] points and the baseline itself missed the 80% floor. SpamAssassin saved 5.04%, but accuracy fell from 88% to 84% with interval [-9, +1] points. Neither test set was released. QS-OCR/Tobacco3482 achieved 70% NL accuracy, SROIE achieved 0.344 field F1 with OCR word-error rate 0.676, and the RVL-CDIP mirror achieved 50% accuracy; all stopped before hybrid validation. These negative outcomes prevent efficient-looking but nonviable configurations from becoming favorable test claims.

Discussion

The main result is not that hybrid workflows always win. It is that PLaND exposes when deterministic substitution is acceptable. LEDGAR retained accuracy within a prespecified engineering margin while eliminating 41.1% of model calls. The mechanism decomposition is important: prompt compression explained only 2.76% of savings, whereas call bypass explained 97.24%. CFPB and SpamAssassin show that lower token use alone is insufficient.

The methodology supports an evidence-driven progression. An unfamiliar task may remain an open-ended agent. Stable steps can move into commands while semantic judgment remains in language. A mature workflow may become a graph, but conversion stops whenever quality declines. References help manage large instructions; they are not automatically deterministic. Runtime guards, fallback, canary monitoring, and automatic demotion are specified design extensions but were not used to obtain these results.

Several assumptions bound interpretation. Dataset labels and selected samples are treated as the evaluation target; the balanced LEDGAR subset does not estimate full multi-label or production prevalence. Tokens and model calls are expense proxies, not measured currency, energy, or carbon. The text experiments test harness-level routing, not autonomous DeepAgent command interpretation. The 2-point margin and 80% floor are task-design choices rather than universal standards. The study uses one local model and machine, most conditions ran once, and it does not directly estimate stochastic variance. Parallel replication timings can vary with warm-up, caching, scheduling, and thermal state and cannot be compared causally with sequential timings. SROIE and RVL test sizes were constrained by eligible source data. Author-designed rules may not transfer to other domains, and the generalized generator/evolver was not itself reevaluated end to end in the post-change replication.

Future work should repeat paired conditions across seeds and models, report case-level output and tool-sequence disagreement, test natural-language and no-skill ablations, and evaluate runtime fallback and demotion. Stateful environments such as tau-bench should use final-state and action verification rather than classification accuracy; PLaND leaves that scorer task-defined.

Conclusion

PLaND is a methodology for selectively moving stable agent work from natural language into tested code under a frozen evaluation contract. On LEDGAR, the accepted hybrid remained within the selected quality margin, reduced tokens by 40.02%, and replaced 411 model calls with commands; an optimized replication reproduced those central findings. Five other workflows stopped at quality gates, showing why savings count only after task performance is protected. The practical rule is simple: keep judgment where it adds value, compile stable operations where evidence permits, and preserve rejection evidence when it does not.

Acknowledgements

The authors thank reviewers of the workflow design and dataset choices. Specific acknowledgements and funding information will be added only with permission.

Disclosure and Conflict of Interest

The authors will provide the journal-required disclosure and conflict-of-interest statement before submission.

Data and Code Availability

Source code, methodology skills, preparation scripts, proof manifests, aggregate results, and paper artifacts are available at https://github.com/mnvsk97/PLaND. Large copyrighted, licensed, or sensitive source records are not redistributed; the repository records their sources, selection procedures, and hashes for authorized reproduction. An archival identifier remains future work.

References

[1] Agent Skills. (2026). *Agent Skills specification*. https://agentskills.io/specification

[2] LangChain. (2026). *DeepAgents: Skills*. https://docs.langchain.com/oss/python/deepagents/skills

[3] LangChain. (2026). *LangGraph overview*. https://docs.langchain.com/oss/python/langgraph/overview

[4] Khattab, O., et al. (2023). DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv*. https://arxiv.org/abs/2310.03714

[5] Li, Z., et al. (2024). AutoFlow: Automated workflow generation for large language model agents. *arXiv*. https://arxiv.org/abs/2407.12821

[6] Zhang, J., et al. (2024). AFlow: Automating agentic workflow generation. *arXiv*. https://arxiv.org/abs/2410.10762

[7] Hu, S., Lu, C., & Clune, J. (2024). Automated design of agentic systems. *NeurIPS*. https://arxiv.org/abs/2408.08435

[8] Yang, Y., et al. (2026). SkillOpt: Executive strategy for self-evolving agent skills. *arXiv*.

https://arxiv.org/abs/2605.23904

[9] Liu, Y., et al. (2026). SkillRevise: Improving LLM-authored agent skills via trace-conditioned skill revision. *arXiv*. https://arxiv.org/abs/2606.01139

[10] Gao, Y., et al. (2026). SkillReducer: Optimizing LLM agent skills for token efficiency. *arXiv*. https://arxiv.org/abs/2603.29919

[11] Kevin, C., et al. (2026). Evaluating skills, not just agents: Agentic continuous evaluation of skills. *arXiv*. https://arxiv.org/abs/2608.20614

[12] Tuggener, D., et al. (2020). LEDGAR: A large-scale multi-label corpus for text classification of legal provisions in contracts. *LREC 2020*, 1235-1241. https://aclanthology.org/2020.lrec-1.155/

[13] Consumer Financial Protection Bureau. (2026). *Consumer Complaint Database*. https://www.consumerfinance.gov/data-research/consumer-complaints/

[14] Apache SpamAssassin. (2006). *Public corpus*. https://spamassassin.apache.org/old/publiccorpus/

[15] Huang, Z., et al. (2021). ICDAR2019 competition on scanned receipt OCR and information extraction. *arXiv*. https://arxiv.org/abs/2103.10213

[16] Harley, A. W., Ufkes, A., & Derpanis, K. G. (2015). Evaluation of deep convolutional nets for document image classification and retrieval. *ICDAR 2015*. https://arxiv.org/abs/1502.07058

[17] Lim, G., Larson, S., & Leach, K. (2024). Label errors in the Tobacco3482 dataset. *arXiv*. https://arxiv.org/abs/2412.13140

[18] LlamaIndex. (2026). *LiteParse: Open-source document parsing*. https://github.com/run-llama/liteparse